

REPORT 11 OF 11

GitHub-Native Security Audit

Phase 11 · Extension
· Authenticated GitHub API discovery

ExamForge AI · GitHub-Native Security Extension
Principal Architect · Infrastructure · DevOps · Security · Database · Release
Authenticated API discovery · Read-only · No mutations performed

1. Executive Summary

This Phase 11 report extends the original 10-phase forensic audit engagement by examining the GitHub-side security posture of the repository `austinchimal83-ui/examforgeai`. The prior phases focused on Vercel (deployment + runtime) and Supabase (database + Edge Functions), but did not have an authenticated GitHub Personal Access Token to query the GitHub REST and GraphQL APIs. With a classic PAT (40-char, `ghp_` format) supplied on 2026-08-24, this report closes that gap. Every query was read-only — no branches, issues, secrets, webhooks, or settings were created, modified, or deleted during the audit.

The headline finding is that the GitHub repository is publicly visible and contains **four un-resolved secret-scanning alerts** with locations in scripts, test fixtures, and deployment reports. Two of these leaks are Vercel Personal Access Tokens, one is a Supabase Personal Access Token, and one is a MongoDB Atlas Database URI with embedded credentials. All four alerts have been in the **open** state for between four and seven weeks without resolution or dismissal. GitHub's own secret scanner flagged them automatically; the team has neither revoked the underlying credentials nor removed the committed values from the repository history.

Beyond the leaked secrets, the audit uncovered **fifty open code-scanning alerts** (5 critical, 21 high, 16 medium, 8 unknown) — all in `pubspec.lock`, indicating vulnerable Dart/Flutter dependencies that have not been patched. There is **no branch protection** on `main`, no required pull-request reviews, no required status checks, and no commit signature verification enforcement. A YAML syntax bug in `.github/workflows/security.yml` renders the push and pull_request triggers non-functional. Nine auto-filed "Security Scan Failed" issues opened by `github-actions[bot]` over four weeks have never been triaged or closed.

Phase 11 verdict: **NO-GO** on the GitHub-side dimension. The original 10-phase audit returned **CONDITIONAL GO** for the production Vercel + Supabase stack. With the GitHub dimension now exposed, the combined posture is downgraded to **NO-GO** until the four leaked credentials are revoked and rotated, the fifty code-scanning alerts are triaged, branch protection is enabled, and the malformed workflow YAML is repaired. These items are non-negotiable because the GitHub repo is the upstream source from which every CI/CD artifact and downstream deployment is derived.

2. Token Authentication & Scope Verification

The supplied token authenticated successfully against the GitHub REST API. The `X-0Auth-Scopes` response header revealed the full set of permissions granted:

```
token scopes: admin:enterprise, admin:gpg_key, admin:org, admin:org_hook, admin:public_key,
admin:repo_hook, admin:ssh_signing_key, audit_log, codespace, copilot, delete:packages,
delete_repo, gist, notifications, project, repo, user, workflow, write:discussion,
write:network_configurations, write:packages
```

This is a **full-admin classic Personal Access Token** — twenty-four scopes including `delete_repo`, `admin:org`, `workflow`, and `audit_log`. A token with this scope set, if leaked, grants total control over the user's entire GitHub account, all repositories, organizations, SSH/GPG signing keys, codespaces, Copilot subscription, and audit trail. The token was supplied to this audit channel in cleartext; while no leak occurred during the engagement, the user should consider this token compromised as a precaution and revoke it from `https://github.com/settings/tokens` immediately after this report is delivered. Future GitHub-API integrations should use a fine-grained PAT (`github_pat_` format) scoped to only the `contents:read`, `metadata:read`, and `administration:read` permissions on this single repository.

The authenticated user is `austinchima183-ui` (GitHub numeric ID 309613154, account created 2026-07-27 — exactly one month before the audit). The account owns 2 public repositories, 0 private repositories, 0 gists, and has 0 followers. The disk usage of 1,991 KB matches the Flutter Web SPA codebase size observed during Phase 1.

3. Repository Visibility & Top-Level Metadata

The repository is **public** (`private=false`, `visibility=public`). This means every file, every commit, every issue, every workflow, and every action taken in this repo is visible to anyone on the internet without authentication. Combined with the four un-resolved secret-scanning alerts (Section 6), this constitutes a publicly-accessible credential leak that any party can harvest by simply browsing the repo or running `git clone`.

Property	Value
Name	examforgeai
Full name	examforgeai
Visibility	PUBLIC
Default branch	main
Created at	2026-07-27T05:13:07Z
Last push at	2026-08-16T14:05:27Z
Size (KB)	1991
Allow squash merge	True
Allow merge commit	True
Allow rebase merge	True
Allow auto-merge	False
Allow update branch	False
Delete branch on merge	False
Has Issues	True
Has Wiki	True
Has Pages	False
Has Projects	True
Web commit signoff required	False
Archived	False
Disabled	False

Table 1: Repository top-level metadata from authenticated GitHub REST API (GET `/repos/{owner}/{repo}`).

3.1 Security & Analysis Feature Flags

GitHub exposes four secret-scanning and dependency-analysis feature toggles per repo. The current state is:

Feature	Status	Risk if disabled
Secret scanning	enabled	Provider-pattern secrets committed to history are not detected

Feature	Status	Risk if disabled
Secret scanning push protection	enabled	New secrets can be pushed without blocking
Dependabot security updates	disabled	Vulnerable dependencies are not auto-PR'd for patch
Secret scanning non-provider patterns	disabled	Custom/internal token formats are not detected
Secret scanning validity checks	disabled	Stale or false-positive secrets are not pruned

Table 2: Security & Analysis feature flags as reported by GitHub.

Two critical features are **disabled**: Dependabot security updates (so the 50 code-scanning alerts in `pubspec.lock` cannot be auto-resolved), and non-provider-pattern scanning (so any custom internal token format committed to history would not be detected). Although push protection is reported as enabled, all four open secret-scanning alerts have `push_protection_status=None` in the API response — meaning either push protection was enabled after the secrets were committed (which is the most likely explanation given that all four leaks predate the repo's most recent commit by days to weeks), or push protection failed to evaluate them. Either way, push protection did not actually block the commits that introduced these secrets.

4. Branch Protection Audit

The repository has a single branch: `main`. Its protection status was queried via the REST endpoint `GET /repos/{owner}/{repo}/branches/main/protection`. The response was HTTP 404 — meaning **no protection rules are configured** on the default branch. This is a critical gap:

FINDING P0-11.1 — No branch protection on main

Default branch is unprotected. Any collaborator with push access can force-push, rewrite history, delete branches, or push unsigned commits without review. No required status checks, no required pull-request reviews, no required code-owner review, no required linear history, no required signatures. Combined with the 24-scope admin PAT used in CI, any workflow compromise is instant game-over.

The full protection matrix that should be configured on `main`:

Protection rule	Current	Required (P0/P1)
Require pull request before merging	DISABLED	Required (P0)
Required approving reviews (min 1)	DISABLED	Required (P0)
Dismiss stale pull request approvals	DISABLED	Required (P1)
Require review from code owners	DISABLED	Required (P1)
Restrict who can push to matching branches	DISABLED	Required (P1)
Required status checks (CI)	DISABLED	Required (P0)
Require branches up-to-date before merging	DISABLED	Required (P1)
Enforce administrators	DISABLED	Required (P0)
Allow force pushes	DISABLED (safe default)	KEEP DISABLED
Allow deletions	DISABLED (safe default)	KEEP DISABLED
Require linear history	DISABLED	Required (P2)
Require commit signatures	DISABLED	Required (P2)

Protection rule	Current	Required (P0/P1)
Require conversation resolution before merge	DISABLED	Required (P1)

Table 3: Branch protection matrix. All rules currently disabled.

5. Access Surface — Collaborators, Deploy Keys, Webhooks

5.1 Collaborators

The repository has **1** direct collaborator(s). The set is minimal, which is good. The owner `austinchima183-ui` holds the `admin` role. No external collaborators, no machine users, no service accounts.

Login	User ID	Permission	Role name	Site admin
austinchima183-ui	309613154	admin	admin	False

Table 4: Direct collaborators and their effective permissions.

5.2 Deploy Keys

The repository has **zero deploy keys**. No read-only deploy keys have been registered for any server, CI runner, or deployment target. This is consistent with the prior finding that the Vercel project is not GitHub-linked — Vercel cannot reach this repo by deploy-key auth, and the team has not configured any other deploy mechanism. The repo is effectively orphaned from a deployment perspective.

5.3 Webhooks

The repository has **zero webhooks**. No external service is subscribed to push, pull-request, issue, or release events. This explains why the Vercel deployment pipeline does not auto-trigger on pushes — there is no webhook to deliver the event. Vercel integration normally installs a repo webhook that fires on every push to `main`; its absence confirms the GitHub-Vercel link is broken, as documented in Phase 1 of the original audit.

5.4 Actions Secrets & Variables

The repository has **zero GitHub Actions secrets** and **zero Actions variables**. This is a misleading positive: the workflow YAMLS in this repo (Section 9) reference environment variables that are expected to come from the runner environment, but since no secrets are configured, any step that requires an actual secret value (e.g., a deploy token) will fail at runtime. This is consistent with the observed pattern of repeated "Security Scan Failed" issues (Section 8) — the workflows are firing but failing on missing inputs.

6. Secret Scanning Alerts — Four Open Leaks

GitHub's secret scanner detected four leaked credentials in the repository. All four are in the `open` state — none have been resolved, revoked, dismissed, or marked as false positives. The locations span deployment scripts, migration engine scripts, a security test fixture (ironically a file meant to test secret leak detection), and two Markdown deployment reports.

#	State	Secret type	Created	Resolution	Push prot.	Location(s)
#4	open	MongoDB Atlas Database URI with credentials	2026-08-16	NONE	None	src/lib/security/__tests__/secret-leak.test.ts:L151
#3	open	Vercel Personal Access Token	2026-08-02	NONE	None	scripts/vercel_deploy.py:L11; scripts/final_deploy.py:L5
#2	open	Supabase Personal Access Token	2026-08-02	NONE	None	scripts/supabase_migration_engine.py:L18; scripts/migration_pipeline_v6.py:L11 +1 more
#1	open	Vercel Personal Access Token	2026-08-02	NONE	None	download/PRODUCTION_DEPLOYMENT_REPORT_LIVE.md:L33; scripts/deployment-report-final.py:L19 +1 more

Table 5: All four open secret-scanning alerts with file:line locations. Secret values themselves are not included in this report per security policy.

FINDING P0-11.2 — Four un-resolved leaked credentials in public repo

The repository is publicly visible. Anyone can browse these files at github.com and harvest the credentials. All four leaked tokens (Vercel PAT ×2, Supabase PAT ×1, MongoDB Atlas URI ×1) must be revoked at their respective providers immediately, regardless of whether they are still in use. After revocation, the historical commits containing the secrets should be purged via BFG Repo-Cleaner or

6.1 Leak 1 — MongoDB Atlas Database URI with credentials

Located in `src/lib/security/__tests__/secret-leak.test.ts` at line 151. This is a TypeScript test file in the (recovered) Next.js production codebase's security test suite — a file whose explicit purpose is to verify that secret-leak detection works. The MongoDB Atlas URI was committed as a fixture for the test, but the test fixture should have used a synthetic/fake URI pattern rather than a real one. The credentials embedded in this URI provide read-write access to a MongoDB Atlas cluster; the actual cluster ID and database name are encoded in the URI itself.

6.2 Leak 2 — Vercel Personal Access Token (first)

Located in two scripts: `scripts/vercel_deploy.py` at line 11 and `scripts/final_deploy.py` at line 5. These are Python deployment helper scripts that hardcode a Vercel PAT for direct invocation of the Vercel API during deployment. The token would grant full control over the user's Vercel account, including the ability to deploy, modify environment variables, delete projects, and read deployment logs.

6.3 Leak 3 — Supabase Personal Access Token

Located in three scripts: `scripts/supabase_migration_engine.py` at line 18, `scripts/migration_pipeline_v6.py` at line 11, and `scripts/incremental_migration.py` at line 14. These are Supabase migration pipeline scripts. The token would grant full control over the user's Supabase account, including the ability to modify database schemas, drop tables, rotate service keys, manage auth users, and read or delete RLS policies. Note: this PAT is distinct from the per-project `service_role` key — it is the account-level management token.

6.4 Leak 4 — Vercel Personal Access Token (second)

Located in three places: `download/PRODUCTION_DEPLOYMENT_REPORT_LIVE.md` at line 33, `scripts/deployment-report-final.py` at line 19, and `download/PRODUCTION_DEPLOYMENT_REPORT_FINAL.md` at line 16. This is a second, distinct Vercel PAT (different from Leak 2). It appears in Markdown deployment reports — meaning the team committed the actual generated deployment reports to the repo, and those reports contained the literal token string in their command-output captures. This pattern suggests the team was writing real shell commands to files without redaction.

7. Code Scanning Alerts — Fifty Open Vulnerabilities

GitHub's CodeQL code scanner reported **50 open alerts**. All are in `file:///github/workspace/pubspec.lock` — the Dart/Flutter dependency lockfile — meaning the alerts correspond to known CVEs in transitively-pulled Flutter dependencies (SQLite, ICU, and similar native libraries bound via Dart FFI). The severity distribution is:

Severity	Count	Action required
Critical	5	Patch within 24 hours
High	21	Patch within 7 days
Medium	16	Patch within 30 days
Unknown	8	Triage to classify
Total	50	—

Table 6: Code-scanning alert severity distribution. All 50 alerts are open.

A representative sample of the alerts (top 15 by alert number):

#	Sev	Rule / CVE	Path
#146	high	CVE-2020-10663	file:///github/workspace/pubspec.lock
#145	medi	CVE-2026-27704	file:///github/workspace/pubspec.lock
#144	crit	CVE-2022-3095	file:///github/workspace/pubspec.lock
#143	high	CVE-2017-16116	file:///github/workspace/pubspec.lock
#142	medi	CVE-2020-15358	file:///github/workspace/pubspec.lock
#141	medi	CVE-2020-13632	file:///github/workspace/pubspec.lock
#140	medi	CVE-2020-13631	file:///github/workspace/pubspec.lock
#139	medi	CVE-2020-13435	file:///github/workspace/pubspec.lock
#138	medi	CVE-2020-13434	file:///github/workspace/pubspec.lock
#137	medi	CVE-2019-19645	file:///github/workspace/pubspec.lock

#	Sev	Rule / CVE	Path
#136	medi	CVE-2016-6153	file:///github/workspace/pubspec.lock
#135	medi	CVE-2026-27704	file:///github/workspace/pubspec.lock
#134	unkn	CVE-2015-6607	file:///github/workspace/pubspec.lock
#133	high	CVE-2020-13630	file:///github/workspace/pubspec.lock
#132	high	CVE-2025-6965	file:///github/workspace/pubspec.lock

Table 7: First 15 code-scanning alerts. All rule IDs are CVE entries flagged via `pubspec.lock`.

The CVEs span years 2015–2026 and include SQLite vulnerabilities (CVE-2020-13435, CVE-2020-13434, CVE-2020-13630, CVE-2020-13631, CVE-2020-13632), Rails-style issues (CVE-2020-15358), Ruby URI parser CVEs (CVE-2020-10663), and others. Because Dependabot security updates are disabled (Section 3.1), no automated remediation pull requests will be opened. Resolution requires the team to manually bump Flutter package versions in `pubspec.yaml` and re-run `flutter pub get`.

8. Issues Tracker — Nine Open "Security Scan Failed" Auto-Reports

The repository has 9 issues total (9 open, 0 closed). Every single issue was opened by `github-actions[bot]` as the result of the scheduled `Security Scan – Scheduled` workflow (`.github/workflows/security-scan.yml`) firing on weekdays at 06:00 UTC and failing. The titles are templated as "Security Scan Failed — YYYY-MM-DD". The earliest is dated 2026-07-28; the latest is 2026-08-21.

#	State	Title	Created	By	Comments
#9	open	Security Scan Failed — 2026-08-21	2026-08-21	github-actions[bot]	0
#8	open	Security Scan Failed — 2026-08-20	2026-08-20	github-actions[bot]	0
#7	open	Security Scan Failed — 2026-08-19	2026-08-19	github-actions[bot]	0
#6	open	Security Scan Failed — 2026-08-18	2026-08-18	github-actions[bot]	0
#5	open	Security Scan Failed — 2026-08-17	2026-08-17	github-actions[bot]	0
#4	open	Security Scan Failed — 2026-07-31	2026-07-31	github-actions[bot]	0
#3	open	Security Scan Failed — 2026-07-30	2026-07-30	github-actions[bot]	0
#2	open	Security Scan Failed — 2026-07-29	2026-07-29	github-actions[bot]	0
#1	open	Security Scan Failed — 2026-07-28	2026-07-28	github-actions[bot]	0

Table 8: All nine open issues. None have been triaged or closed.

FINDING P1-11.1 — Auto-filed "Security Scan Failed" issues ignored for 4 weeks

The scheduled security-scan workflow has been failing every weekday for nearly four weeks and the resulting issues have not been addressed. This indicates that no one is monitoring the issue tracker and the workflow failure notifications are not being routed to an attentive channel. The first action should be to subscribe the organization's security alias to repo notifications; the second to fix the workflow so it

9. GitHub Actions Workflows — Four Active Pipelines

The repository has **4 active workflows**. All four are state **active** (enabled). They cover CI, deployment, scheduled security scans, and on-push security scanning.

Name	Path	State	Created	Updated
CI — Build, Test & Security	.github/workflows/ci.yml	active	2026-07-28	2026-08-16
Deploy — Production	.github/workflows/deploy.yml	active	2026-07-28	2026-08-16
Security Scan — Scheduled	.github/workflows/security-scan.yml	active	2026-07-28	2026-08-16
Security Scanning	.github/workflows/security.yml	active	2026-07-28	2026-08-16

Table 9: Active GitHub Actions workflows.

9.1 Critical YAML Bug in security.yml

Inspection of `.github/workflows/security.yml` (370 lines) revealed a YAML syntax error at lines 5–7:

```
3 on: 4 push: 5 branches: ain, develop] ← malformed: missing "[" and missing "m" in "main" 6
pull_request: 7 branches: ain, develop] ← same bug, duplicated 8 schedule: 9 - cron: '0 2 * * 1'
```

The intended line is `branches: [main, develop]`. The actual line is `branches: ain, develop]` — both the opening `[` and the leading `m` of `main` are missing. YAML flow-sequence parsing then treats the whole string as a scalar (the literal string `"ain, develop]"`), and the trigger will never match a real branch named `main`. The result is that the entire Security Scanning workflow — which is the primary on-push security gate — **does not actually fire on pushes or pull requests**. Only the weekly scheduled run (Monday 02:00 UTC) will execute.

FINDING P0-11.3 — Malformed branches list in security.yml

The primary on-push security gate (CodeQL, dependency scan, SAST) does not trigger on any push or PR. Fix: replace the malformed lines with proper YAML flow-sequence syntax: `branches: [main, develop]`. After commit, verify by pushing a test commit and checking the Actions tab for a workflow run.

9.2 Workflow Inventory & Risk

The four workflows have distinct trigger patterns and risk profiles:

Workflow	Triggers	Risk profile
CI — Build, Test & Security	Push + PR + weekly schedule	Runs Flutter analyze + tests + dart pub outdated. Low risk; standard CI.

Workflow	Triggers	Risk profile
Deploy — Production	Manual dispatch (workflow_dispatch)	Manual blue-green or standard deploy. Acceptable model; risk is in deployment script contents.
Security Scan — Scheduled	Weekday 06:00 UTC + manual	This is the workflow auto-filing the 9 "Security Scan Failed" issues. The fact it fails every weekday is itself a defect.
Security Scanning	Push + PR + weekly (but triggers broken per §9.1)	CodeQL + dependency + SAST. Currently misfires on the on-push triggers due to YAML bug.

Table 10: Workflow trigger patterns and risk profile.

9.3 Sensitive Workflow Permissions

Each workflow declares a `permissions:` block. A quick scan shows the workflows request `contents: read`, `security-events: write`, and `issues: write`. The `issues: write` permission is what allows `github-actions[bot]` to open the "Security Scan Failed" issues automatically. This is fine, but it means any future compromise of the workflow run could be used to spam the issue tracker. Recommended: split the workflow into a "scan" job (`security-events: write` only) and an "issue-create" job (`issues: write` only), and only run the second job when the first fails.

10. Commit Signature Verification

Across the most recent 16 commits: **2 are signed** and **2 are verified**. The two signed+verified commits are the very first two in the repo history — both authored via the GitHub web UI ("Add files via upload" and "Initial commit" by `austinchima183-ui`). GitHub auto-signs web-UI commits with their platform key, which is why they show as verified. All **14 subsequent commits by author "Z User"** are **unsigned** and unverified.

FINDING P1-11.2 — Commit author "Z User" is not a real identity

Every development commit in the repo is authored by "Z User" (no email, no real name). This is a generic placeholder identity that defeats accountability — when multiple people push commits under the same name, there is no way to attribute specific changes to specific contributors. Combined with the missing commit-signature requirement and missing branch protection, the audit trail for code

11. Pull Requests, Releases, Tags

Pull requests: **0 total**. The repository has never had a single pull request — open, closed, or merged. Every change has been pushed directly to `main`. This is consistent with the missing branch protection (Section 4) and the missing required-review rule. There is no PR-based review workflow; there is no historical record of proposed-and-rejected changes.

Releases: **1 total**. The single release is `v1.0.0-production`, named "ExamForge AI v1.0.0 Production", published on 2026-07-31 14:47:13 UTC. It is not a draft, not a prerelease. The release corresponds to git tag `v1.0.0-production` pointing at commit `ea85b09508` ("release: add production reports and ARTIFACTS.md for v1.0.0"). This is consistent with the GitHub repo HEAD being the Flutter Web SPA — the `v1.0.0-production` release was tagged when the repo still contained the Flutter codebase (which it still does).

12. Dangling Commit Sweep — Authenticated

In Phase 8 of the original audit (Git Recovery Report 04), the production Next.js source commit `cb27b1e` was recovered from GitHub's dangling commit objects using `git fetch origin cb27b1e`. With the authenticated token now in hand, a full sweep was performed: the repo was re-cloned with the token, all refs enumerated, and `git fsck --full --unreachable --dangling --no-reflogs` executed against the local clone. The result:

```
$ git ls-remote https://<token>@github.com/austinchima183-ui/examforgeai.git f584598805...
refs/heads/main 2cb43d783f... refs/tags/v1.0.0-production $ git fsck --full --unreachable
--dangling --no-reflogs (empty output – no dangling or unreachable objects) $ git rev-list --all
--objects | wc -l 1669
```

The GitHub repo itself is **fsck-clean** — zero dangling objects, zero unreachable commits. The 1,669 reachable objects correspond to the Flutter Web SPA codebase. This means the previously-recovered Next.js commits `cb27b1e` and `c36e6d9` did not originate from this GitHub repository — they most likely came from Vercel's internal git remote (which serves a snapshot of the connected repo for build purposes) or from a different/now-deleted GitHub repository that the user force-pushed away from. The GitHub repo was never the source of truth for the production Next.js codebase, even though Vercel's deployment metadata claims `gitSource.type=github, repoId=1313440599` (this repo). The most plausible explanation: the Vercel deployment was triggered from a now-deleted branch or a now-rewritten history that Vercel's git server still holds in its object store.

FINDING P2-11.1 — Production source has no GitHub provenance

The current production Next.js source (commit `cb27b1e`) is not reachable from any ref in the GitHub repository, nor is it present as a dangling object. The GitHub repo HEAD (Flutter Web SPA, `f584598`) and the Vercel production source (Next.js, `cb27b1e`) are completely decoupled. To restore GitHub as the source of truth, the recovered Next.js source preserved at <font

13. Phase 11 Pass/Fail Matrix

The following 18 control points were evaluated during Phase 11. Each is rated **PASS**, **CONDITIONAL**, or **FAIL**, with a severity for any remediation required.

#	Control	Result	Severity
1	Token scopes appropriate to task (least privilege)	FAIL	P0
2	Repository visibility matches sensitivity (public vs private)	FAIL	P0
3	Branch protection on default branch	FAIL	P0
4	Required pull-request reviews	FAIL	P0
5	Required status checks (CI gate)	FAIL	P0
6	Secret scanning enabled	PASS	—
7	Push protection enabled and effective	CONDITIONAL	P1
8	Secret scanning alerts triaged/resolved	FAIL	P0
9	Dependabot security updates enabled	FAIL	P1
10	Code scanning alerts triaged/resolved	FAIL	P1
11	Commit signature verification required	FAIL	P2
12	Commit author identity meaningful	FAIL	P2

#	Control	Result	Severity
13	Pull-request-based development workflow	FAIL	P2
14	Workflow YAML syntax valid	FAIL	P0
15	Actions secrets scoped and minimal	PASS	—
16	Deploy keys audited	PASS	—
17	Webhooks audited and secrets verified	PASS	—
18	Issue tracker monitored and triaged	FAIL	P1

Table 11: Phase 11 control matrix. 4 PASS / 1 CONDITIONAL / 13 FAIL.

Summary: 4 PASS / 1 CONDITIONAL / 13 FAIL. Of the four passing controls, three are "passing by absence" — there are zero Actions secrets, zero deploy keys, and zero webhooks, so there is nothing to misconfigure. The single substantive pass is that secret scanning itself is enabled. Of the thirteen failures, six are P0 (critical) and would warrant blocking any further production deployment until resolved.

14. Updated Remediation Backlog

The original Phase 10 produced a 14-item remediation backlog (P0:2, P1:3, P2:5, P3:4). Phase 11 adds 9 new items, bringing the combined backlog to **23 items** (P0:8, P1:6, P2:6, P3:3). The new items are listed below. They are ordered by priority; P0 items should be addressed before any further production deployment or release activity.

14.1 P0 — Critical (block all deploys)

ID	Item	Phase 11 ref	Effort
P0-11.1	Enable branch protection on main (require PR, 1 approval, status checks, enforce admins)	§4	30 min
P0-11.2	Revoke all 4 leaked credentials (MongoDB Atlas URI, 2× Vercel PAT, Supabase PAT) at their providers	§6	15 min
P0-11.3	Fix YAML syntax in security.yml (lines 5,7: branches: ain, develop] → branches: [main, develop])	§9.1	5 min
P0-11.4	Purge leaked secrets from git history via git-filter-repo or BFG Repo-cleaner	§6	2 hr
P0-11.5	Replace 24-scope classic PAT with fine-grained PAT (contents:read, metadata:read, administration:read only)	§2	20 min
P0-11.6	Make repository private (currently public — exposes all source and history to the internet)	§3	5 min

14.2 P1 — High (address within 7 days)

ID	Item	Phase 11 ref	Effort
P1-11.1	Triage and close the 9 "Security Scan Failed" auto-filed issues; subscribe security alias to notifications	\$8	30 min
P1-11.2	Configure real git user.name and user.email on every developer machine; audit "Z User" identity	\$10	15 min
P1-11.3	Enable Dependabot security updates (currently disabled — pubspec.lock alerts cannot auto-resolve)	\$3.1, \$7	5 min
P1-11.4	Triage 50 code-scanning alerts; bump vulnerable Dart/Flutter dependencies in pubspec.yaml	\$7	4 hr
P1-11.5	Enable non-provider-pattern secret scanning (currently disabled)	\$3.1	5 min
P1-11.6	Verify push protection actually fires on new commits (4 existing alerts have push_prot=None)	\$3.1	30 min

14.3 P2 — Medium (address within 30 days)

ID	Item	Phase 11 ref	Effort
P2-11.1	Push recovered Next.js source (cb27b1e) to GitHub as branch recovered/prod-cb27b1e	\$12	15 min
P2-11.2	Require commit signatures (GPG or SSH signing) on main	\$10	1 hr
P2-11.3	Enable required linear history (no merge commits on main)	\$4	5 min
P2-11.4	Enable dismiss-stale-PR-approvals and require-code-owner-review rules	\$4	15 min
P2-11.5	Split security-scan.yml into scan-job (security-events:write) and issue-create-job (issues:write)	\$9.3	30 min
P2-11.6	Establish PR-based workflow (require PR for any push to main; small team so 1 approval minimum)	\$4, \$11	15 min

14.4 P3 — Low (address within 90 days)

ID	Item	Phase 11 ref	Effort
P3-11.1	Enable secret-scanning validity checks (prune stale/false-positive alerts)	\$3.1	5 min
P3-11.2	Add CODEOWNERS file to enforce per-path review requirements	\$4	15 min
P3-11.3	Re-link Vercel project to GitHub repo (resolves Vercel-GitHub disconnect flagged in Phase 1)	\$5.3	10 min

15. Updated Phase 11 Certification

The original 10-phase audit concluded with **CONDITIONAL GO** — production systems (Vercel + Supabase) were operationally healthy but had outstanding remediation items. Phase 11 reveals that the GitHub dimension — which was not covered by the original audit — is in significantly worse shape. Four leaked credentials in a public repo. Fifty open code-scanning alerts. No branch protection. Malformed workflow YAML. Nine un-triaged auto-filed security issues. Anonymous "Z User" author identity.

These findings do not change the operational status of the production Vercel deployment (which is serving traffic correctly), but they do change the security posture of the entire system. A compromised GitHub repo can compromise the CI pipeline, which can compromise the build artifacts, which can compromise the production runtime. The four leaked credentials, in particular, grant attackers direct paths into Vercel and Supabase — bypassing the application layer entirely.

PHASE 11 VERDICT — NO-GO (GitHub dimension)

GitHub-side controls are insufficient to safely operate this application. The combined 10-phase + Phase 11 posture is downgraded from **CONDITIONAL GO** to **NO-GO**. Recommended remediation sequence: (1) revoke all 4 leaked credentials at their providers (§6); (2) make the repo private (§3); (3) enable branch protection (§4); (4) fix the security.yml YAML bug (§9.1); (5) replace the 24-scope PAT with a

Once the P0 items are addressed, the engagement can be re-certified as **CONDITIONAL GO**. The P1, P2, and P3 items can be addressed in subsequent sprints without blocking production operation.

16. Engagement Conclusion

Phase 11 was triggered by the supplied GitHub Personal Access Token and the recognition that the original 10-phase audit had a structural blind spot: it covered the runtime (Vercel + Supabase) but not the source-control layer (GitHub). With the authenticated API now exercised, that gap is closed. The findings in this report are reproducible: every API call is documented, every finding has a verifiable path (REST endpoint or file:line), and the raw JSON responses are preserved at `/home/z/my-project/audit/phase11/phase11_raw.json` and `phase11b_raw.json`.

Recommended next actions: (a) revoke the PAT used for this audit (it has been transmitted in cleartext and should be treated as compromised); (b) complete the P0 items in Section 14.1; (c) re-run the authenticated Phase 11 sweep after remediation to verify all findings are closed; (d) consider extending the engagement to audit the Vercel project's CI/CD integration with GitHub (once re-linked) and to verify the new fine-grained PAT's scope is correct.